

On the Compressibility of Quantized Large Language Models

Yu Mao*, Weilan Wang*, Hongchao Du*, Nan Guan* and Chun Jason Xue†

*City University of Hong Kong

Email: {yumao7-c, weilawang2-c, hongcdu2-c}@my.cityu.edu.hk, nanguan@cityu.edu.hk

†MBZUAI Email: jason.xue@mbzuai.ac.ae

Abstract—Deploying Large Language Models (LLMs) on edge or mobile devices offers significant benefits, such as enhanced data privacy and real-time processing capabilities. However, it also faces critical challenges due to the substantial memory requirement of LLMs. Quantization is an effective way of reducing the model size while maintaining good performance. However, even after quantization, LLMs may still be too big to fit entirely into the limited memory of edge or mobile devices and have to be partially loaded from the storage to complete the inference. In this work, we take a preliminary step of studying applying data compression techniques to reduce data movement and thus speed up the inference of quantized LLM on memory-constrained devices. In particular, we discussed the compressibility of quantized LLMs, the trade-off between the compressibility and performance of quantized LLMs, and opportunities to jointly optimize both of them.

Index Terms—Large Language Model, Data compression, Zstandard, Quantization, Memory

I. INTRODUCTION

Constrained memory becomes a bottleneck for Large Language Models (LLMs) deployments [1]–[5]. For example, most mobile devices are equipped with 4-12 GB of memory, which hinders running large models effectively. Some approaches have been proposed to alleviate this burden, including quantization [6]–[9], activation recomputation [10]–[12], and offloading [1], [2], [13]. In this paper, we provide another perspective that has been overlooked in solving this problem.

In 2016, DeepCompression [14] introduced a quantization method for compressing models and also applied Huffman coding to the quantized weights. However, given the model sizes at that time, entropy coding had a negligible effect. In DeepCompression, Huffman coding achieved a mere 1.2x compression ratio in addition to the 4x ratio from quantization. Consequently, entropy coding has been largely overlooked in later developments of model compression techniques [6]–[9].

Large-language models create opportunities for entropy coding due to increased redundancy in the model. This paper shows that by applying Int8 quantization and a lossless entropy coder to large-language models, it is possible to compress **model weights by 8x** and **activations by 16x** without compromising accuracy. Quantization alone provides a 4x compression, while the added entropy coding contributes an additional 2x for weights and 4x for activations.

Entropy coding’s introduction is challenging due to its reliance on prior quantization methods, which affect both compression ratios and model accuracy. This paper examines how

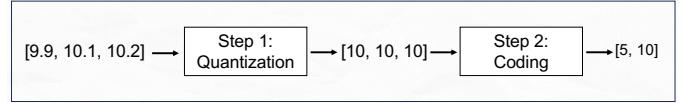


Figure 1: A complete lossy compression process involves two steps: quantization and entropy coding. The latter has long been overlooked in current model compression research. For OPTs, INT8 quantization achieves 4x compression ratio, while entropy coding achieves 2x for weights and 4x for activations.

quantization affects a model’s compressibility and accuracy, with entropy as the linking factor. Assuming the effective knowledge is uniformly distributed across a matrix, selecting a quantization method to enhance compression typically results in reduced accuracy of the quantized model. This trade-off arises as higher compression entails information loss, which is inevitable given the uniform distribution of information. The hypothesis is tested by comparing two common quantization methods with varying distributions after quantization.

Fortunately, effective information often exhibits locality in the matrix. Recent research on large language models (LLMs) [6], [9], [15]–[17] has found that some channels in their matrices, known as outliers, are more crucial for accuracy than others. The key to simultaneously achieving high accuracy and compressibility in quantized models lies in preserving outlier information while reducing non-outlier information. We explore an LLM quantization technique that preserves outlier data while reducing non-outlier data, resulting in similar accuracy and higher compression ratios compared to LLM quantization methods that preserve all data informations.

II. RELATED WORK

Quantization. Quantization reduces LLM model size. LLM.int8([9]) uses 8-bit vector-wise quantization and mixed INT8/FP16 precision. AWQ [15] finds weights’ importance for LLMs’ performance. ZeroQuant [16] uses token-wise activation quantization and group-wise weight quantization. Smoothquant [6] smooths activation magnitudes for better quantization. Outlier Suppression+ [17] extends understanding of asymmetric distribution in outliers on specific channels.

Modern Data Compression Techniques. Modern compression techniques like Deflate [18], Gzip [19], and Zstandard [20] combine Huffman [21] and LZ [22] algorithms with

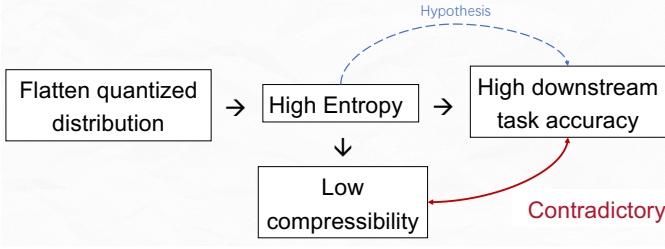


Figure 2: The relationship between quantized weights / activations, entropy, accuracy, and compressibility. Achieving high accuracy and high compressibility is contradictory in theory.

unique designs. FSE [23] is a new kind of entropy encoder based on ANS theory that achieves high-speed compression accuracy. Recently, neural-network-based compressors [24]–[26] have achieved remarkable ratios but unsatisfactory speeds. Therefore this paper only focuses on simple entropy coders.

III. LLM’S COMPRESSIBILITY AND ACCURACY

The section defines information entropy and its relation to data compressibility, then proposes a conjecture based on recent research [27]–[29] that suggests higher accuracy in quantized models leads to lower compressibility when the effective information is evenly distributed in matrix. An experimental comparison of tensor-wise and channel-wise quantization methods is provided to justify this conjecture.

A. Compressibility and Accuracy

1) *Compressibility and Entropy*: In information theory, the entropy of a random variable is the average level of “information” inherent to the variable’s possible outcomes [30].

Definition III.1. Given a discrete random variable X , which takes values in the alphabet $\mathcal{X}$ and is distributed according to $p : \mathcal{X} \rightarrow [0, 1]$, the entropy is

$$H(X) := - \sum_{x \in \mathcal{X}} p(x) \log p(x) \quad (1)$$

where Σ denotes the sum of the variable’s possible values.

Theorem III.1. The entropy of data increases with the uniformity of its distribution, reaching a maximum when the distribution is completely uniform. [30].

Entropy measures the unpredictability of data; higher entropy indicates greater difficulty in prediction and compression.

Theorem III.2. Higher entropy leads to lower compressibility and vice versa [30].

2) *The relationship between quantized model accuracy and entropy*: This section explores the connection between quantized model accuracy and entropy. We need to prioritize the assumption that information is uniformly distributed across the matrix, representing an ideal scenario. This means every piece of data within the matrix holds equal importance and equally influences model accuracy. Our goal is to ensure all

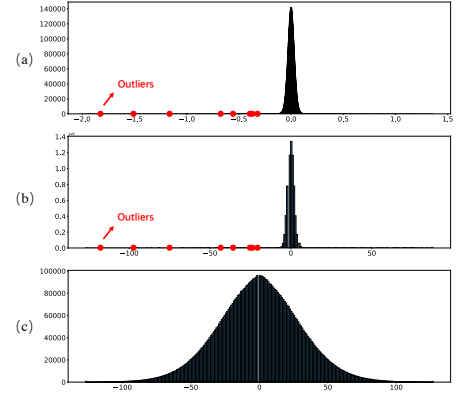


Figure 3: Weight distributions of the original data matrix, tensor-wise and channel-wise quantized matrix.

data information is fully preserved. Consequently, a higher entropy in the quantized matrix signifies more knowledge, which translates into greater accuracy for the quantized model. However, by definition, increased entropy also results in reduced compressibility.

Conjecture III.1. With the given assumption, higher matrix entropy leads to improved model accuracy after quantization.

Based on the above conjecture, we make a claim about the relation between accuracy and compressibility.

Corollary III.2.1. With the given assumption, the higher compressibility of the quantized model indicates lower quantized model accuracy.

We re-emphasize that the model compression method outlined in this paper involves two steps: **quantization and entropy coding**. The impact of different quantization methods on entropy coding performance will be discussed in the upcoming sections. Note that all compression ratios reported in subsequent sections are in addition to a 4x reduction from Int8 quantization, which means the total compression ratio is $4 \times C$, where C is the additional compression ratio achieved by entropy coding.

B. Experimental Justification

The comparison of tensor-wise and channel-wise quantization is used as an instance to justify the above corollary with experimental results. Tensor-wise and channel-wise quantization are two commonly used model quantization methods. Tensor-wise quantization selects max/min values for the entire matrix range, resulting in uneven quantized distribution. Channel-wise quantization selects max/min values for each row/column, resulting in a more flatten quantized distribution.

Channel-wise quantized matrices have more information but are less compressible, while tensor-wise quantized matrices have less information but are more compressible due to different distribution characteristics. Fig. 3 shows the distribution of `mlp.c_proj.weight` in the GPT2-large model. Fig. 3(a) shows the non-uniform distribution of original floating-point

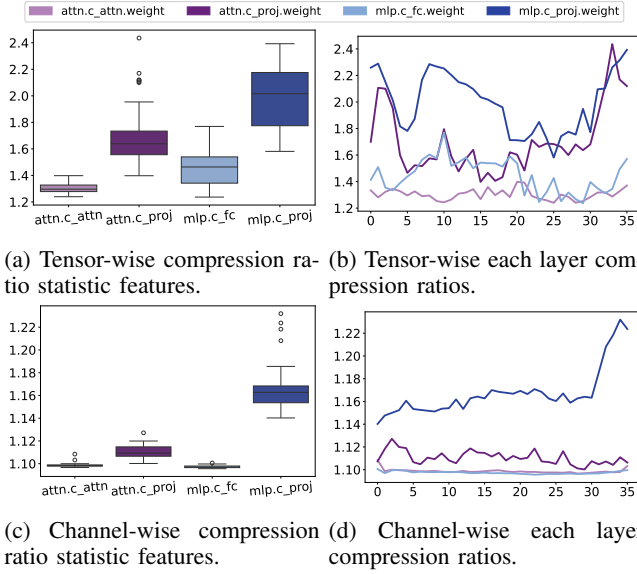


Figure 4: Compression ratios on tensor-wise and channel-wise quantized model.

numbers, including outliers. Fig. 3(b) depicts the leptokurtic distribution of weights after tensor-wise quantization, with most values confined to -10 to 10. In contrast, channel-wise quantization results in a platykurtic distribution shown in Fig. 3(c). This contributes to higher weight entropy and reduced compressibility (Section 4.1).

Naive tensor-wise and channel-wise quantization methods are implemented on GPT2-large model to show the conclusion. The overall compression ratio is shown in Tab. I. We can conclude that the channel-wise quantization indeed has lower compressibility than tensor-wise quantization with higher accuracy, therefore demonstrating our conclusion.

Table I: Layer Average Entropy Coding Compression Ratio in GPT2-large.

Avg. CR.	attn.c_attn	attn.c_proj	mlp.c_fc	mlp.c_proj
Tensor-wise	1.09	1.11	1.09	1.17
Channel-wise	1.31	1.71	1.45	1.99

We further give out the detailed compression performance statistics of the two quantization methods. From Fig. 4, we can draw the following conclusions:

- Under per-tensor quantization, model weights can still be compressed to 1.2x to 2.4x after INT8 quantization.
- Different layers have varying compression ratios, such as *attn.c_attn.weight* averaging at 1.3 and *mlp.c_proj.weight* being at least 1.6 or higher.
- The first and last layers generally have higher compression ratios compared to the middle layer.
- Channel-wise quantization reduces following compressibility significantly, resulting in a compression rate of only 1.1 to 1.2 times.

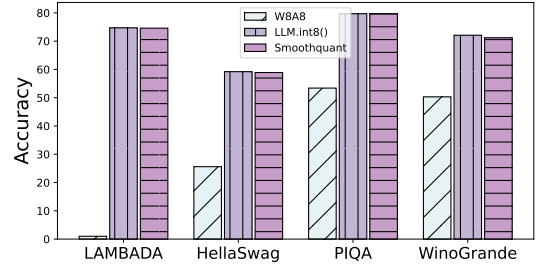


Figure 5: Accuracy on four downstream tasks of W8A8 (Tensor-wise), Smoothquant (Tensor-wise), and LLM.int8() (Channel-wise).

IV. OPPORTUNITIES FOR ATTAINING BOTH METRIC EXCELLENCE

The statement in Sec. III-A2 suggests a challenging outcome: as accuracy improves, compressibility diminishes. We will discuss the opportunities to attain good performance on both accuracy and compressibility. Specifically, by employing SmoothQuant, the cutting-edge LLM quantization technique, we show that with careful design, it's possible to simultaneously attain impressive results in both areas.

A. Outlier-aware LLM Quantization Method

The opportunity to achieve good performance on both compressibility and quantized LLM accuracy lies in leveraging the effective information locality. The assumption in Sec. III-A2 that information is uniformly distributed across the matrix may not hold, as effective information often shows local patterns in current Large Language Models (LLMs). An example of this is the presence of outliers. Recent studies have shown that not all channels in large language models (LLMs) are equally important [6]–[9]. Specifically, only a select few channels within each matrix—referred to as outliers—have a significant impact on the model's performance. In contrast, the remaining channels, deemed non-outliers, contribute minimally. We suggest that by selectively quantizing data, prioritizing important information and compressing less critical data, we can simultaneously attain high compression rates and maintain accuracy.

Smoothquant, an outlier-aware tensor-wise quantization method for LLMs, matches the accuracy of channel-wise quantization on downstream tasks (Fig. 5) and maintains a high entropy coding compression ratio (Tab II). Smoothquant operates by dividing input activation with a per-channel smoothing factor, and to keep the calculations equivalent, it multiplies the weights by the inverse of this smoothing factor.

$$Y = (X \text{diag}(s)^{-1}) \cdot (\text{diag}(s)W) = \hat{X}\hat{W} \quad (2)$$

Such smoothing factor is calculated from values of weights and activations.

$$s_j = \frac{\max(|X_j|)^\alpha}{\max(|W_j|)^{1-\alpha}} \quad (3)$$

Table II: Layer Average Entropy Coding Compression Ratio in OPTs.

Avg. CR.	OPT-1.3b	OPT-6.7b	OPT-13b
Tensor-wise Activation	4.32	4.35	4.98
Smoothquant Activation	3.14	3.23	3.16
LLM.int8() Activation	1.84	1.87	1.87
Tensor-wise Weight	1.54	1.36	1.32
Smoothquant Weight	1.64	1.52	1.46
LLM.int8() Weight	1.16	1.14	1.13

This step transfers a portion of the activation numeric values to the weights, ensuring that outliers maintain a larger value in both the activation and weight matrices. This ensures that during quantization, these outliers and their corresponding weights preserve maximum information by receiving more effective bits due to their larger original values. For example, when an outlier activation is 100 and its weight is 1, the operation adjusts both values to 10. Given that 10 is still high relative to typical activations (the original outlier is over 100 times larger than non-outliers) and weights (usually between 0 and 1), post-Int8 quantization will push them closer to the maximum value of 127, resulting in more effective bits.

Tab. II shows the average compression ratio for LLM.int8(), Smoothquant, and tensor-wise quantization on OPT models. For the activation processed by Smoothquant, the amount of information increased significantly (the compression rate increased by 25%~30%), while the amount of weight information only increased by ~7%. Another interesting phenomenon is that compared to the compression rate of weight which is only about 1.5x, the compressibility of activation is obviously better and can reach 5x.

V. EXPERIMENTS

To further extend our conclusion to practical LLM application scenarios, this section explore the compression effect on a widely utilized LLM series, OPT models [31] under two state-of-the-art LLM quantization methods, Smoothquant (INT8, *tensor-wise*) [6] and LLM.int8() (INT8, *channel-wise*) [9].

A. Compression Ratio Under Different Entropy Coders

Huffman-Coding [21], FSE Coding [23], and Zstandard [20] are selected as compression baselines. As illustrated, under the state-of-the-art compression method, the model after tensor-wise Smoothquant has a compression rate of more than 1.5 times, while the channel-wise LLM.int8() quantized weight only has a compression rate of 1.1~1.3 times.

B. Efficiency Report

Compression Speed. Three compression methods were compared in terms of their speeds. FSE coding had the slowest speed, especially for decompression. Huffman Coding had a compression speed twice as fast as FSE coding and a decompression speed of 1.3G per second. Zstd achieved the best overall performance with the fastest compression

Table III: Compression Ratio for Overall Quantized Model under Different Entropy Coders.

SmoothQuant	OPT-1.3b	OPT-6.7b	OPT-13b	OPT-30b
Zstd	2.22	1.78	1.66	1.69
FSE	2.11	1.77	1.66	1.68
Huffman	2.15	1.76	1.66	1.69
LLM.int8()	OPT-1.3b	OPT-6.7b	OPT-13b	OPT-30b
Zstd	1.3	1.2	1.18	1.11
FSE	1.31	1.2	1.18	1.11
Huffman	1.3	1.2	1.18	1.11

and decompression speeds, while maintaining a satisfactory compression rate. Its decompression speed was twice that of Huffman coding at 2.3GB per second (Tab. IV).

Table IV: Entropy Coding and Decoding Speed.

Speed(MB/s)	zstd	FSE	Huffman
Comp. Speed	710	325	600
Decomp. Speed	2300	440	1350

Model Loading Time Reduction in Real-World Scenario.

To show the effect of compression on model loading, we measure loading time before and after using compression. We deploy the experiments on a testbed with an 8-core Intel Core i7-9700 CPU (3.00Ghz) and 64GB DDR4 DRAM. A 512GB KIOXIA XG6 SSD is utilized for storage with performance up to 3180 MB/s and 2800 MB/s for sequential read and write, respectively. Table V shows the results. Mmap performs at half the level of read system call due to 4KB parameter loading and limited storage bandwidth utilization. Compression reduces loading time by 41-55% with mmap and 40-60% with read, similar to using multiple threads for loading time reduction.

Table V: Loading Time Before and After Entropy Coding.

Time(s)	OPT-1.3b	OPT-6.7b	OPT-13b	OPT-30b
mmap-before	1.02	4.52	8.45	19.17
mmap-after	0.46	2.51	5.0	11.32
read-before	0.55	2.46	4.6	10.52
read-after	0.25	1.39	2.76	6.28

VI. CONCLUSION

The paper explores the use of compression to reduce data volume in large language model (LLM) calculations, finding that quantized LLMs are highly compressible. The level of quantization granularity influences this compressibility by altering weight distribution, which is analyzed using information theory. Experiments with two advanced LLM quantization methods confirm these results. Furthermore, a loading experiment shows that compression can double the data transfer rate between flash memory and DRAM.

REFERENCES

- [1] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” in *International Conference on Machine Learning*, pp. 31094–31116, PMLR, 2023.
- [2] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, “Llm in a flash: Efficient large language model inference with limited memory,” *arXiv preprint arXiv:2312.11514*, 2023.
- [3] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 16344–16359, 2022.
- [4] L. Zhang, X. Liu, Z. Li, X. Pan, P. Dong, R. Fan, R. Guo, X. Wang, Q. Luo, S. Shi, *et al.*, “Dissecting the runtime performance of the training, fine-tuning, and inference of large language models,” *arXiv preprint arXiv:2311.03687*, 2023.
- [5] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, and R. McHardy, “Challenges and applications of large language models,” *arXiv preprint arXiv:2307.10169*, 2023.
- [6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *International Conference on Machine Learning*, pp. 38087–38099, PMLR, 2023.
- [7] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [8] S.-y. Liu, Z. Liu, X. Huang, P. Dong, and K.-T. Cheng, “Llm-fp4: 4-bit floating-point quantized transformers,” *arXiv preprint arXiv:2310.16836*, 2023.
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Llm. int8 (): 8-bit matrix multiplication for transformers at scale,” *arXiv preprint arXiv:2208.07339*, 2022.
- [10] V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro, “Reducing activation recomputation in large transformer models,” *Proceedings of Machine Learning and Systems*, vol. 5, 2023.
- [11] P. Jain, A. Jain, A. Nrusimha, A. Gholami, P. Abbeel, J. Gonzalez, K. Keutzer, and I. Stoica, “Checkmate: Breaking the memory wall with optimal tensor rematerialization,” *Proceedings of Machine Learning and Systems*, vol. 2, pp. 497–511, 2020.
- [12] S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhunoye, G. Zerveas, V. Korthikanti, *et al.*, “Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model,” *arXiv preprint arXiv:2201.11990*, 2022.
- [13] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, “{ZeRO-Offload}: Democratizing {Billion-Scale} model training,” in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pp. 551–564, 2021.
- [14] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” *arXiv preprint arXiv:1510.00149*, 2015.
- [15] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “Awq: Activation-aware weight quantization for llm compression and acceleration,” *arXiv preprint arXiv:2306.00978*, 2023.
- [16] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “Zeroquant: Efficient and affordable post-training quantization for large-scale transformers,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27168–27183, 2022.
- [17] X. Wei, Y. Zhang, Y. Li, X. Zhang, R. Gong, J. Guo, and X. Liu, “Outlier suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling,” *arXiv preprint arXiv:2304.09145*, 2023.
- [18] P. Deutsch, “Deflate compressed data format specification version 1.3,” tech. rep., 1996.
- [19] P. Deutsch, “GZIP file format specification version 4.3,” *RFC*, vol. 1952, pp. 1–12, 1996.
- [20] Y. Collet, “Zstd github repository from facebook,” 2016.
- [21] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [22] J. Ziv and A. Lempel, “A universal algorithm for sequential data compression,” *IEEE Transactions on information theory*, vol. 23, no. 3, pp. 337–343, 1977.
- [23] J. Duda, “Asymmetric numeral systems: entropy coding combining speed of huffman coding with compression rate of arithmetic coding,” *arXiv preprint arXiv:1311.2540*, 2013.
- [24] Y. Mao *et al.*, “Accelerating general-purpose lossless compression via simple and scalable parameterization,” in *MM’22*, pp. 3205–3213, 2022.
- [25] Y. Mao *et al.*, “Trace: A fast transformer-based general-purpose lossless compressor,” in *WWW*, pp. 1829–1838, 2022.
- [26] Y. Mao, J. Li, Y. Cui, and J. C. Xue, “Faster and stronger lossless compression with optimized autoregressive framework,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2023.
- [27] J. Rae, “Compression for agi,” tech. rep., 2023.
- [28] G. Delétang, A. Ruoss, P.-A. Duquenne, E. Catt, T. Genewein, C. Matern, J. Grau-Moya, L. K. Wenliang, M. Aitchison, L. Orseau, *et al.*, “Language modeling is compression,” *arXiv preprint arXiv:2309.10668*, 2023.
- [29] Y. Yu, S. Buchanan, D. Pai, T. Chu, Z. Wu, S. Tong, B. Haeffele, and Y. Ma, “White-box transformers via sparse rate reduction,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [30] C. E. Shannon, “A mathematical theory of communication,” *The Bell system technical journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [31] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.